Estructuras de datos. Clasificaciones.

- *Según la implementación:*
  - **Provistas por los lenguajes: Básicas.** por ej. Arrays, Strings, Structs, Files.
  - **Abstractas: TDA.**

Tipo de Dato Abstracto.

Un TDA (Tipo de Dato Abstracto) es un tipo de dato *definido por el programador* que se puede manipular de un modo similar a los tipos de datos provistos por el lenguaje.

Está formado por un *conjunto de valores válidos de datos* y un *conjunto de operaciones primitivas* que se pueden realizar sobre esos valores.

Los TDA proporcionan un mecanismo adicional mediante el cual se realiza una separación clara entre la *interfaz* y la *implementación* del tipo de dato.

- La *implementación* de un TDA consta de:
  - La *representación*: elección de las estructuras de datos.
  - Las *operaciones*: elección de los algoritmos.
- La *interfaz* del TDA se asocia con las operaciones y datos del TDA y es visible al exterior

Tener en cuenta que:

- **El usuario [creador del TDA] define:**
  - Cómo va a representar el nuevo tipo de datos que está creando.
  - Cuáles van a ser las operaciones primitivas (*interfaz*).
  - El código (*implementación*) de dichas operaciones.
- El programador ["cliente"] que utilice el TDA debe conocer la *representación* y la *interfaz*, pero la *implementación* queda *oculta* para el.

Implementación de TDA en C++

- Una de las características de C++ que permite implementar TDA son los *archivos de inclusión o cabecera*, utilizados para agrupar en ellos variables externas, declaraciones de datos comunes y prototipos de funciones.
- Los archivos de cabecera se incluyen (mediante la directiva al preprocesador #include) en los *archivos que contienen la codificación de las funciones*, *archivos fuente* y también en los *archivos de código que hagan referencia a algún elemento del archivo de inclusión*.
- Al implementar un TDA en C++ se agrupa en este archivo la representación de los datos y la interfaz del TDA (representada por los prototipos de las funciones).

Consideraciones de uso de TDA en C++

- Los programas fuente que vayan a utilizar un TDA necesitan conocer los *datos públicos* del mismo, osea su *representación* y las *funciones primitivas* que se tienen, por ello deben tener un #include "nombre_tda.h". [El h es de "header"].
- La implementación de las funciones primitivas es *transparente* para el programa que usa el TDA, si bien debe existir y ensamblarse, el programador no necesita conocer ese código.

El rol de la Abstracción

- La abstracción es la capacidad para *encapsular y aislar información* del diseño y ejecución (ocultamiento de información).
- La abstracción ha sido clave como mecanismo para poder *controlar problemas de mayor complejidad*.
- Ante la dificultad de dominar en su totalidad los objetos complejos, se ignoran detalles no esenciales, tratando en su lugar con el *modelo ideal* del objeto y centrándonos en el estudio de sus aspectos esenciales.

Ventajas de los TDA

- Permite una *mejor conceptualización y modelización* del mundo real. Mejora la representación y comprensibilidad. Clarifica los objetos basados en estructuras y comportamientos comunes.
- Mejora la *robustez* del sistema. Los TDA permiten la comprobación de tipos para evitar errores en el tiempo de ejecución.
- Separa la *implementación* de la *especificación*. Permite la modificación y mejora la implementación sin afectar la interfaz pública del TDA.
- Permite la *extensibilidad del sistema*. Los componentes de software reusables son más fáciles de crear y mantener.